چالش ۲ از ۳ — «پشت میز DBA»

مرحله: دوم (پیش‌نیاز: خروجی مرحله ۱ — پیش‌نیاز مرحله ۳ و چالش نهایی)

محور: PostgreSQL، Index، Function، View، Query Optimization، Execution Plan، Performance و Procedure، اتصال Python به Database

زمان پیشنهادی: ۵ تا ۷ روز کاری

سقف زمان اجرای نوتبوک: ۱۰ دقیقه (بدون زمان بارگذاری اولیه داده)

فایل 00_Delivery_Guide را خوانده‌اید؟ شرایط تحویل همان است.

صورت مسئله در یک نگاه

یک فروشگاه اینترنتی بزرگ (داده‌های واقعی و عمومی Olist برزیل) تمام سفارش‌های دو سال خود را در ۹ جدول مرتبط منتشر کرده است: سفارش، آیتم‌های سفارش، مشتری، فروشنده، محصول، پرداخت، نظر مشتری، موقعیت جغرافیایی، و ترجمه دسته محصولات.

تیم محصول چند سؤال تحلیلی دارد و همه آن‌ها را روی یک داشبورد می‌خواهد که هر بار باز شدن داشبورد، کوئری‌ها زنده اجرا می‌شوند — نتیجه از پیش آماده‌شده روی دیسک نیست.

تیم یک شرط گذاشته است: هیچ کوئری داشبورد نباید بیش از ۲۰۰ میلی‌ثانیه طول بکشد. این عدد، بودجه شماست و قابل مذاکره نیست.

شما DBA این سیستم هستید. مسئله این است:

داده خام و پراکنده را به یک Database درست‌طراحی‌شده تبدیل کنید، سؤال‌های تیم محصول را به کوئری بدل کنید، و هر کوئری را زیر بودجه زمانی تعیین‌شده بیاورید — با شاهد، نه با حدس.

«با شاهد» یعنی: برای هر بهبود، باید پلن اجرای قبل و بعد را نشان دهید و بگویید چرا عوض شد. اگر کوئری سریع شد و نمی‌دانید چرا، آن بهبود نمره ندارد.

منابع داده

منبع اصلی — Olist Brazilian E-Commerce

- Kaggle: olistbr/brazilian-ecommerce
- حدود ۱۰۰٬۰۰۰ سفارش، ۹ فایل CSV مرتبط، بازه ۲۰۱۶ تا ۲۰۱۸.
- داده واقعی است: تاریخ‌های خالی، سفارش‌های بدون نظر، متن‌های پرتغالی، و رکوردهایی که به هم وصل نمی‌شوند.

منبع دوم — خروجی مرحله ۱

خروجی تمیزِ کارخانه مرحله ۱ هم باید در همین Database، در یک schema جداگانه، بارگذاری شود.

الزام زنجیره: بارگذاری داده‌ها باید از طریق همان package مرحله ۱ انجام شود، با افزودن یک مقصد جدید (Database) به کارخانه. کد بارگذاری یک‌بارمصرف و جداگانه قبول نیست. اگر معماری مرحله ۱ اجازه افزودن مقصد جدید را نمی‌دهد، بازطراحی‌اش بخشی از این مرحله است.

آن‌چه باید تولید شود

۱) طراحی schema

- جدول‌ها، نوع ستون‌ها، کلید اصلی، کلید خارجی و محدودیت‌های لازم (NOT NULL, CHECK, UNIQUE) را خودتان طراحی کنید. کپی مستقیم ساختار CSVها قبول نیست.
- برای هر انتخاب نوع داده که غیربدیهی است، دلیل بنویسید.
- داده‌ای که محدودیت‌ها را نقض می‌کند قبل از بارگذاری تعیین تکلیف شود، نه با حذف محدودیت.
- گزارش دهید چند رکورد از داده خام به دلیل نقض یکپارچگی ارجاعی (foreign key) قابل بارگذاری نبود و با آن‌ها چه کردید.

۲) پنج سؤال تیم محصول

این پنج سؤال را باید به کوئری SQL تبدیل کنید. صورت سؤال، طرح کوئری نیست — تصمیم‌های پیوند، فیلتر، سطح تجمیع و رفتار با مقادیر خالی با شماست:

1. سبد و درآمد: در هر ماه، به تفکیک دسته محصول، ارزش کل فروش، تعداد سفارش یکتا، و میانگین ارزش هر سفارش چقدر بوده است؟
2. تحویل: چه نسبتی از سفارش‌ها دیرتر از تاریخ تخمینیِ وعده‌داده‌شده تحویل شده‌اند، و این نسبت به تفکیک ایالت مشتری و ماه چگونه تغییر کرده است؟
3. رضایت: رابطه میان تأخیر تحویل و امتیاز نظر مشتری چیست؟ خروجی باید جدولی باشد که این رابطه را به شکل قابل‌خواندن نشان دهد.

4. فروشنده: ۲۰ فروشنده برتر بر اساس درآمد چه کسانی‌اند، و برای هرکدام میانگین زمان آماده‌سازی سفارش و میانگین امتیاز نظر چقدر است؟

5. پرداخت: توزیع روش‌های پرداخت و تعداد اقساط چگونه است و آیا با ارزش سفارش ارتباطی دارد؟

برای هر کوئری:

- بودجه: زمان اجرا باید زیر ۲۰۰ میلی‌ثانیه باشد — برای همه پنج کوئری، نه اکثرشان.
- اگر برای یکی به بودجه نرسیدید، تحویل مردود نمی‌شود، اما باید دقیقاً و با شاهد پلن بگویید چه چیزی مانع است، چه چیزهایی را امتحان کردید، و چرا به نتیجه نرسید. «وقت نشد» توضیح نیست.
- روش اندازه‌گیری زمان باید روشن و تکرارپذیر باشد. یک‌بار اجرا و خواندن عدد، اندازه‌گیری نیست.

۳) چرخه بهینه‌سازی با شاهد

برای هر پنج کوئری، این چرخه باید در نوتبوک مستند باشد:

1. نسخه اولیه کوئری + زمان اجرا + پلن اجرا
2. خواندن پلن: کدام بخش گران است؟ Cost و تعداد سطر تخمینی در مقابل واقعی چه می‌گویند؟ کجا Sequential Scan می‌شود و کجا Index Scan؟
3. تغییری که اعمال می‌کنید — انتخاب نوع تغییر با شماست و بخشی از مسئله است
4. زمان و پلن بعد از تغییر
5. جمله‌ای که می‌گوید چرا این تغییر کار کرد

قاعده سخت: برای هر Index که می‌سازید باید توجیه با شاهد پلن داشته باشید. Index بدون شاهد، نمره منفی دارد — چون در سیستم واقعی هزینه دارد.

۴) هزینه پنهان Index (پیچیدگی اضافه‌شده)

Index مجانی نیست. باید اندازه بگیرید و گزارش کنید:

- حجم هر Index که ساخته‌اید و مجموع آن نسبت به حجم جدول.
- اثر Index روی نوشتن: سرعت درج یک دسته بزرگ رکورد قبل و بعد از ساخت Indexها را اندازه بگیرید و مقایسه کنید.
- در پایان، یک جمع‌بندی: آیا همه Indexهایی که ساختید ارزشش را داشتند؟ حداقل یک Index را که ساختید و بعد تصمیم گرفتید حذفش کنید، با دلیل گزارش کنید.

۵) کوئری‌ای که Index نجاتش نمی‌دهد (پیچیدگی اضافه‌شده)

دست‌کم یکی از کوئری‌های شما باید به حالتی برسد که ساختن Index، هر Indexای، آن را به بودجه نرساند — و برای رسیدن به بودجه مجبور شوید به چیزی جز Index فکر کنید.

اگر پنج سؤال بالا طبیعتاً شما را به چنین حالتی نرساندند، یک سؤال ششم از خودتان طراحی کنید که این وضعیت را بسازد.

این کوئری هم در پایان باید زیر بودجه باشد. در گزارش باید بنویسید:

- چرا Index این‌جا بی‌اثر بود و پلن اجرا این بی‌اثری را از کجا نشان می‌داد،
- چه چیزی در صورت‌بندی مسئله مانع استفاده Database از Index می‌شد،
- چه کردید و چرا جواب داد،
- و چه چیزی را در عوضش پرداختید.

این بخش، مرز میان «کسی که Index می‌سازد» و «کسی که Plan می‌خواند» را نشان می‌دهد.

۶) View و استفاده مجدد

- حداقل دو View: یکی ساده روی یک جدول برای پنهان کردن پیچیدگی فیلترها، و یکی چندجدولی که منطق پیوندِ تکرارشده را یک‌جا جمع کند.
- حداقل یکی از پنج کوئری باید روی View نوشته شود، نه جدول‌های خام.
- تصمیم View یا جدول snapshot (پیچیدگی اضافه‌شده): برای سنگین‌ترین تحلیل، دو راه را بسازید — یک View که هر بار محاسبه می‌کند، و یک جدول از پیش محاسبه‌شده. هر دو را زمان بگیرید و با عدد بگویید کدام را برای داشبورد انتخاب می‌کنید و چه چیزی را از دست می‌دهید (تازگی داده؟ فضا؟ پیچیدگی نگه‌داری؟).

۷) Procedure و Function

- یک Stored Procedure با پارامتر بنویسید که snapshot تحلیلی یک ماه مشخص را بسازد یا بازسازی کند.
- باید idempotent باشد: اجرای دوباره آن برای یک ماه، داده را دوباره اضافه نکند.
- رفتارش در برابر خطا در میانه کار باید تعریف‌شده باشد: داده نباید نیمه‌کاره بماند.
- یک Function بنویسید که یک مقدار محاسبه‌شده معنادار برگرداند و بتوان آن را داخل SELECT به‌کار برد.
- در گزارش، تفاوت Function و Procedure را با شاهد از کد خودتان توضیح دهید: چه چیزی را با یکی می‌شود کرد و با دیگری نمی‌شود، و چرا هرکدام را آن‌جا انتخاب کردید. توضیح کتابی بدون ارجاع به کد خودتان، نمره ندارد.

۸) پل Python و SQL

تمام تعامل نوتبوک با Database از طریق SQLAlchemy باشد، با این شرایط:

- مدیریت صریح Connection؛ هیچ Connection بازمانده‌ای نباید پس از اجرای نوتبوک باقی بماند.
- هیچ کوئری‌ای با چسباندن رشته ساخته نشود. پارامترها باید پارامتر باشند. این مورد سخت‌گیرانه بررسی می‌شود.
- خواندن نتیجه یک کوئری بزرگ به DataFrame باید دسته‌ای انجام شود، نه یک‌جا.
- نوشتن یک DataFrame به Database با نگاشت نوع ستون‌های مشخص‌شده خودتان انجام شود، نه نوع‌های پیش‌فرض خودکار.
- خواندن از View هم باید نشان داده شود.

۹) تست و Git

- حداقل ۶ تست برای لایه داده: درستی مقدار بازگشتی idempotent ،Function بودن Procedure، و صحت نتیجه حداقل یک کوئری تحلیلی روی یک داده کوچک و کنترل‌شده.
- همه DDL (ساخت جدول، Function، Procedure، View، Index) باید در فایل‌های .sql نسخه‌دار در گیت باشد. ساختاری که فقط دستی در pgAdmin ساخته شده و در repo نیست، وجود ندارد.
- حداقل ۸ commit معنادار.

معیار پذیرش این مرحله

- ☐ با یک فرمان، از یک PostgreSQL خالی تا Database کامل و پرشده بالا بیاید.
- ☐ هر پنج کوئری (و کوئری ششم، اگر ساختید) زیر بودجه ۲۰۰ms باشند، با روش اندازه‌گیری تکرارپذیر.
- ☐ کوئری بخش ۵ هم زیر بودجه باشد، اما از راهی جز ساختن Index.
- ☐ برای هر کوئری، پلن قبل و بعد در نوتبوک موجود باشد.
- ☐ برای هر Index، توجیه با شاهد وجود داشته باشد.
- ☐ اثر Index روی سرعت نوشتن اندازه‌گیری شده باشد.
- ☐ Procedure دوبار اجرا شود و داده تکراری نسازد.
- ☐ هیچ کوئری ساخته‌شده با الحاق رشته در کد نباشد.

پیش‌نمایش مرحله بعد

در مرحله ۳ مدل یادگیری ماشین می‌سازید و داده‌اش را از همین Database می‌خوانید، نه از CSV. پس هر چیزی که این‌جا ساخته‌اید — Viewها، Procedure، نوع ستون‌ها، سرعت کوئری — آن‌جا زیر بار می‌رود. اگر خواندن داده کند باشد، هر چرخه آزمایش مدل کند می‌شود و خودتان اول از همه اذیت می‌شوید.